

1. Чтобы узнать имя текущего пользователя используем команду: `whoami`

```
eltex-v21@eltex-2026-summer:~$ whoami
```

```
eltex-v21
```

Далее, чтобы запустить сессию `screen` с именем `eltex-v21_part2` и включить логирование, используем команду:

```
screen -S "$(whoami)_part2" -L -Logfile "${USER}_part.log"
```

Команда создаёт сессию `screen` с именем `eltex-v21_part2` и записывает вывод команд в файл `eltex-v21_part.log`.

После попадаем в сессию и выполняем команды для уменьшения вероятности появления управляющих ESC-последовательностей и цветного форматирования в лог-файле.

```
eltex-v21@eltex-2026-summer:~$ export TERM=dumb
eltex-v21@eltex-2026-summer:~$ alias ls='ls --color=never'
eltex-v21@eltex-2026-summer:~$ PS1='$ '
$ whoami
eltex-v21
$ echo "$USER"
eltex-v21
$ █
```

Команда: `export TERM=dumb`

Задаёт простой тип терминала без расширенного форматирования.

Команда: `alias ls='ls --color=never'`

отключает цветной вывод команды `ls`

Команда: `PS1='$ '`

Делает приглашение командной строки простым, чтобы в лог-файл не попадали лишние символы оформления.

Далее выполняем проверку команд `whoami` и `echo "$USER"`. Они подтверждают, что работа выполняется от пользователя `eltex-v21`.

Отсоединяемся от сессии и проверяем созданную сессию

```
eltex-v21@eltex-2026-summer:~$ screen -S "$(whoami)_part2" -L -Logfile "${USER}_part.log"
[detached from 731984.eltex-v21_part2]
eltex-v21@eltex-2026-summer:~$ screen -ls
There is a screen on:
      731984.eltex-v21_part2  (06.07.2026 21:13:41)  (Detached)
1 Socket in /run/screen/S-eltex-v21.
```

2. В командной строке введем `ls` после нажатием дважды `tab` появятся доступные команды начинающейся на `ls`.

```

ls      lsattr    lsblk    lsb_release  lscpu    lsfd    ls
initramfs  lsipc    lsirq    lslocks     lslogins  lsmem   lsmo
d        lsns     lsof     lspci       lsusb
$ ls

```

3. Введем в командную строку `echo $HIST`

```

$ echo $HIST
$HISTCMD      $HISTCONTROL  $HISTFILE     $HISTFILESIZE $HISTSIZ
$ echo $HIST

```

Для автодополнения переменных введена команда `echo $HIST`. Символ `$` указывает Bash, что дополняется имя переменной.

4. `$ echo $HISTFILESIZE`

2000

Это означает, что в файле истории может храниться до 2000 команд.

5. Для вывода скрытых файлов и каталогов напишем команду: `ls -d ~/.*`

```

$ ls -d ~/.*
/home/eltex-v21/.bash_history /home/eltex-v21/.bash_logout /home/eltex-v21/.ba
shrc /home/eltex-v21/.config /home/eltex-v21/.lessht /home/eltex-v21/.profil
e /home/eltex-v21/.viminfo
$

```

Символ `~` обозначает домашний каталог пользователя, а шаблон `.*` выводит скрытые файлы и каталоги.

6. Для вывода даты и времени выполнения команд в истории была задана переменная `HISTTIMEFORMAT`:

```
export HISTTIMEFORMAT="%d.%m.%Y %H:%M:%S "
```

После этого была выполнена команда `history`. В результате история команд стала отображаться с датой и временем выполнения команд.

```

67 06.07.2026 21:14:41 alias ls='ls --color=never'
68 06.07.2026 21:15:10 PS1='$ '
69 06.07.2026 21:15:52 whoami
70 06.07.2026 21:16:10 echo "$USER"
71 06.07.2026 22:27:46 ls
72 06.07.2026 23:05:27 echo $HISTFILESIZE
73 06.07.2026 23:06:09 echo $HISTSIZ
74 06.07.2026 23:15:33 ls -d ~/.*
75 06.07.2026 23:34:06 export HISTTIMEFORMAT="%d.%m.%Y %H:%M:%S "
76 06.07.2026 23:34:24 history
$

```

7. Для проверки автоматического сохранения истории сначала была выполнена команда:

date

После этого была проверена история текущей сессии командой:

```
history | tail
```

и файл истории команд:

```
tail ~/.bash_history
```

Затем была задана переменная PROMPT_COMMAND:

```
export PROMPT_COMMAND='history -a'
```

Команда history -a дописывает новые команды из кэша истории в файл ~/.bash_history.

После настройки была выполнена команда:

```
whoami
```

Повторная проверка history | tail и tail ~/.bash_history показала, что команда whoami появилась как в кэше истории, так и в файле истории.

```

$ date
Пн 06 июл 2026 23:42:45 +07
$ history | tail
 69 06.07.2026 21:15:52 whoami
 70 06.07.2026 21:16:10 echo "$USER"
 71 06.07.2026 22:27:46 ls
 72 06.07.2026 23:05:27 echo $HISTFILESIZE
 73 06.07.2026 23:06:09 echo $HISTSIZE
 74 06.07.2026 23:15:33 ls -d ~/.*
 75 06.07.2026 23:34:06 export HISTTIMEFORMAT="%d.%m.%Y %H:%M:%S "
 76 06.07.2026 23:34:24 history
 77 06.07.2026 23:42:45 date
 78 06.07.2026 23:43:14 history | tail
$ tail ~/.bash_history
ls
vi report_part1.txt
exit
watch /usr/bin/vmstat
screen -ls
screen -r report
screen -ls
screen -S 321673.top -X quit
screen -ls
logout
$ export PROMPT_COMMAND='history -a'
$ whoami
eltex-v21
$ history | tail
 73 06.07.2026 23:06:09 echo $HISTSIZE
 74 06.07.2026 23:15:33 ls -d ~/.*
 75 06.07.2026 23:34:06 export HISTTIMEFORMAT="%d.%m.%Y %H:%M:%S "
 76 06.07.2026 23:34:24 history
 77 06.07.2026 23:42:45 date
 78 06.07.2026 23:43:14 history | tail
 79 06.07.2026 23:43:47 tail ~/.bash_history
 80 07.07.2026 00:27:39 export PROMPT_COMMAND='history -a'
 81 07.07.2026 00:28:41 whoami
 82 07.07.2026 00:28:55 history | tail
$ tail ~/.bash_history
#1783356194
history | tail
#1783356227
tail ~/.bash_history
#1783358859
export PROMPT_COMMAND='history -a'
#1783358921
whoami
#1783358935
history | tail
$ 

```

10. Для создания переменной DATE с текущей датой была использована команда:

```
DATE=$(date +%d.%m.%Y)
```

Проверка содержимого переменной:

```
echo $DATE
```

```

$ DATE=$(date +%d.%m.%Y)
$ echo $DATE
07.07.2026
$ 

```

11. Что бы создать TIME с текущим временем была использована команда:

```
TIME=$(date +%H:%M:%S)
```

Проверка содержимого переменной:

```
echo $TIME
```

Команда date +%H:%M:%S выводит текущее время, а конструкция \$(...) подставляет результат команды в переменную TIME.

```
$ TIME=$(date +%H:%M:%S)
$ echo $TIME
00:48:59
$
```

12. Для создания переменной DATE_TIME были объединены значения переменных DATE и TIME:

```
DATE_TIME="$DATE $TIME"
```

Проверка содержимого переменной:

```
echo "$DATE_TIME"
```

В результате была выведена дата и время, разделённые пробелом.

```
$ DATE_TIME="$DATE $TIME"
$ echo "$DATE_TIME"
07.07.2026 00:48:59
$
```

13. Для вывода имён файлов из каталогов /bin и /sbin, содержащих хотя бы одну цифру, была использована команда:

```
ls /bin/[0-9] /sbin/[0-9]
```

Шаблон [0-9] выбирает имена файлов, в которых есть хотя бы одна цифра.

```
$ ls /bin/[0-9]* /sbin/[0-9]*
/bin/addr2line      /bin/gcov-12        /bin/ping6          /bin/sha512sum      /bin/x86_64-linux-gnu-gcc-nm-12  /bin/x86_64-linux-gnu-ld.gold  /sbin/e2undo
/bin/b2sum          /bin/gcov-dump-12   /bin/pl2pm          /bin/ssh-argv0      /bin/x86_64-linux-gnu-gcc-ranlib  /bin/x86_64-linux-gnu-lto-dump  /sbin/e4crypt
/bin/base32         /bin/gcov-tool-12   /bin/pod2html       /bin/systemd-1d128  /bin/x86_64-linux-gnu-gcc-ranlib-12 /bin/x86_64-linux-gnu-nm       /sbin/e4defrag
/bin/base64         /bin/grub-mkpasswd-pbkdf2 /bin/pod2man       /bin/traceroute6.db /bin/x86_64-linux-gnu-gcov        /bin/x86_64-linux-gnu-objcopy   /sbin/fsck.ext2
/bin/bunzip2        /bin/grub-syslinux2cfg /bin/pod2text       /bin/x86_64-linux-gnu-ar /bin/x86_64-linux-gnu-gcov-12     /bin/x86_64-linux-gnu-objdump  /sbin/fsck.ext3
/bin/bzip2          /bin/hzxs           /bin/pod2usage      /bin/x86_64-linux-gnu-addr2line /bin/x86_64-linux-gnu-gcov-dump-12 /bin/x86_64-linux-gnu-objdump  /sbin/fsck.ext4
/bin/bzip2recover   /bin/hzxs           /bin/py3clean       /bin/x86_64-linux-gnu-gcov-dump-12 /bin/x86_64-linux-gnu-gcov-tool   /bin/x86_64-linux-gnu-ranlib   /sbin/killall5
/bin/c89            /bin/hzxs           /bin/py3compile     /bin/x86_64-linux-gnu-gcov-tool /bin/x86_64-linux-gnu-gcov-tool-12 /bin/x86_64-linux-gnu-readelf  /sbin/mke2fs
/bin/c89-gcc        /bin/i386           /bin/py3versions    /bin/x86_64-linux-gnu-gold /bin/x86_64-linux-gnu-gcov-tool-12 /bin/x86_64-linux-gnu-size     /sbin/mkfs.ext2
/bin/c99            /bin/linux32        /bin/pydoc3         /bin/x86_64-linux-gnu-gold /bin/x86_64-linux-gnu-gprof       /bin/x86_64-linux-gnu-strings  /sbin/mkfs.ext3
/bin/c99-gcc        /bin/linux64        /bin/pydoc3.11      /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprof       /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/cpan5.36-x86_64-linux-gnu /bin/lto-dump-12   /bin/pygettext3     /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/cpp-12         /bin/md5sum         /bin/pygettext3.11  /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/diff3          /bin/md5sum.textutils /bin/pythons        /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/enc2xs         /bin/pdb3           /bin/pythons.11     /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/gcc-12         /bin/pdb3.11        /bin/sha1sum        /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/gcc-ar-12      /bin/perls.36.0     /bin/sha224sum      /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/gcc-nm-12      /bin/perls.36.x86_64-linux-gnu /bin/sha256sum     /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
/bin/gcc-ranlib-12  /bin/ping4          /bin/sha384sum      /bin/x86_64-linux-gnu-gprof /bin/x86_64-linux-gnu-gprofng     /bin/x86_64-linux-gnu-strip   /sbin/mkfs.ext4
$
```

14. Для изменения приглашения командной строки была изменена переменная PS1:

```
PS1='$(whoami)@$(hostname)-$(date +%H:%M)> '
```

В приглашении стали отображаться имя пользователя, имя хоста и текущее время.

```
/bin/gcc-ranlib-12 /bin/ping4
$ PS1='$(whoami)@$(hostname)-$(date +%H:%M)> '
eltex-v21@eltex-2026-summer-01:03>
```

15. Для передачи установленного приглашения в запускаемый интерпретатор `bash` была экспортирована переменная `PS1`:

```
export PS1
```

Затем был запущен новый интерпретатор `bash` без чтения файлов настроек:

```
bash --noprofile --norc
```

Проверка командой `echo "$PS1"` показала, что значение переменной `PS1` сохранилось. В новом `bash` приглашение осталось в формате:

```
имя_пользователя@имя_хоста-НН:ММ>
```

```
eltex-v21@eltex-2026-summer-01:11> export PS1
eltex-v21@eltex-2026-summer-01:11> bash --noprofile --norc
eltex-v21@eltex-2026-summer-01:11> echo "$PS1"
$(whoami)@$(hostname)-$(date +%H:%M)>
eltex-v21@eltex-2026-summer-01:15> █
```

16. Для создания в домашнем каталоге подкаталогов для каждого месяца текущего года была использована команда:

```
mkdir -p ~/"$(date +%Y)"-{01..12}
```

Команда `date +%Y` подставляет текущий год, а конструкция `{01..12}` создаёт значения месяцев от 01 до 12.

Проверка была выполнена командой:

```
ls -d ~/"$(date +%Y)"-*
```

```
eltex-v21@eltex-2026-summer-01:15> mkdir -p ~/"$(date +%Y)"-{01..12}
eltex-v21@eltex-2026-summer-01:20> ls -d ~/"$(date +%Y)"-*
/home/eltex-v21/2026-01 /home/eltex-v21/2026-03 /home/eltex-v21/2026-05 /home/eltex-v21/2026-07 /home/eltex-v21/2026-09 /home/eltex-v21/2026-11
/home/eltex-v21/2026-02 /home/eltex-v21/2026-04 /home/eltex-v21/2026-06 /home/eltex-v21/2026-08 /home/eltex-v21/2026-10 /home/eltex-v21/2026-12
eltex-v21@eltex-2026-summer-01:21> █
```